Middleware Architecture — IS3108 / SCS3203

Quick Study Notes · SriBank PLC Scenario

SriBank PLC — 100+ branches, 500 ATMs, VISA/MasterCard — building Internet & Mobile Banking. Must integrate with **4 heterogeneous backend systems**. Retail customers only; 2-factor authentication required.

Backend Systems

System	Protocol	Purpose
Core Banking System	RESTful API	Customer KYC, accounts, balances, transaction history
Inter-bank Payment Gateway	ISO 8583	Route payments across local banks (Central Bank SL)
Credit Card System	SOAP	Card limits, balances, billing cycle, active status
Internal Payment Switch	TCP/IP POX XML	Links to MasterCard / VISA international processing

---

Q1 Software Architecture Design

Proposed: 4-Tier Architecture + Service-Oriented Architecture (SOA)

Tier	Logical Layer	Key Components
1 · Client / Presentation	Presentation Layer	React/Angular SPA, iOS & Android apps, client-side validation, push notification handler, API client
2 · Business Logic	Business Logic + Service Layer	API Gateway (auth, rate-limit, routing), Retail Banking, Account Mgmt, Special Request, Security/Identity (2FA + JWT), Notification Orchestration Services
3 · Integration	Integration + Adapter Layer	Message Broker (Kafka/RabbitMQ), REST/ISO 8583/SOAP/POX Adapters, Workflow Engine, Push (APNS/FCM) + SMS Delivery Services
4 · Data	Data Storage Layer	RDBMS for sessions, audit logs, workflow states, user prefs. External DBs accessed only via Integration Tier.

Architectural Patterns

◆ N-Tier Architecture

- Each tier scales independently — e.g. scale mobile frontend without touching the DB.
- Security boundary: clients never reach Data Tier directly.
- Shared business logic for both web and mobile — no duplication.

◆ Service-Oriented Architecture (SOA)

- Functions exposed as services: BalanceInquiry, FundsTransfer, LoanRequest, etc.
- Same services reused by Internet Banking AND Mobile Banking.
- Integration Tier adapters shield Business Logic from backend protocol complexity.

◆ Publish-Subscribe (Pub/Sub)

- Events (e.g. FundsTransferCompleted) published to Message Broker — publisher does not wait.
- Notification Service subscribes and delivers push/SMS asynchronously.

- Loan/credit requests use async workflow — not real-time, multi-step approvals.

◆ **Client-Side MVC**

- Model: data + validation rules. Controller: handles input. View: renders UI.
- Instant client-side validation — no server round-trip for basic checks.
- Partial DOM updates only — no full page reload, feels fast and fluid.

Q2 Client-Side Architecture — MVC / MVVM

Best Pattern: Client-Side MVC (Single Page Application via React, Angular, or Vue.js)

Variations: MVVM (Angular), MVP are also acceptable.

Component	Role	Banking Example
Model	Holds data + validation rules	Account number regex, amount > 0 rule, balance state
Controller	Handles input, calls API, updates View	On Transfer click: validate → call API → refresh View
View	Renders UI, shows errors immediately	Red border + error message shown without page reload

Benefits

- **Instant Validation:** Controller checks Model rules before any server call. User sees feedback in milliseconds.
- **No Full Page Reloads:** Only JSON travels after initial load. View updates only the changed DOM section.
- **Consistent State:** Model is single source of truth. Balance updates everywhere after a transfer — automatically.
- **Testable:** Model validation logic is unit-testable independently from the UI.

Trade-offs in a Banking Context

■ **Security**	Client code is inspectable. All validation MUST be repeated server-side. Never store tokens in localStorage (XSS risk). Use HTTP-only cookies for refresh tokens.
■ **Initial Load**	Full JS bundle on first visit. Mitigate with code splitting, lazy loading, CDN caching.
■ **Complexity**	Requires skilled frontend developers. Data-binding bugs can be harder to debug than server-rendered pages.

Q3 Integration Architecture & EIPs

Primary Pattern: Integration Layer (ESB / Integration Microservices)

A dedicated tier with protocol-specific adapters handles all external system calls. Business Logic never calls backends directly — if a backend changes, only its adapter needs updating.

3 Essential Enterprise Integration Patterns (EIPs)

SPLITTER	**What:** Breaks a composite message into individual messages for separate processing.
EXAMPLE	**SriBank:** User submits 3 utility bill payments at once. Splitter breaks into 3 separate messages, each routed independently to the Inter-bank Payment Gateway. Failure of one does not block others.

CONTENT-BASED ROUTER	**What:** Routes a message to a different destination based on its content.
EXAMPLE	**SriBank:** Funds Transfer: reads toAccountBankIdentifier — internal SriBank account → Core Banking Adapter; local external bank → Inter-bank Gateway Adapter; international → Internal Payment Switch Adapter.
AGGREGATOR	**What:** Collects responses from multiple systems and merges into one reply.
EXAMPLE	**SriBank:** Account Summary: queries Core Banking (deposit balance) AND Credit Card System (card balance) in parallel. Both tagged with same correlation ID. Aggregator waits for both, merges into one AccountSummaryResponse.

Q4 Notification Architecture — Push & SMS

(a) Synchronous vs Asynchronous

	Synchronous ✗	Asynchronous ✓
How it works	Sender waits for delivery confirmation before continuing	Sender publishes event, moves on immediately
Banking problem	Transfer blocked if notification service is slow or down	Transaction confirms instantly; notification sent in background
Scalability	1000 tx/s = 1000 waits → system freezes	Messages queue in broker; processed at notification service pace
Reliability	One failure cascades — whole flow can break	Broker persists messages; retried automatically on failure

(b) Best Pattern: Publish-Subscribe (Pub/Sub)

(c) How Pub/Sub ensures reliability & scalability under volatile conditions

◆ **Message Persistence**	Broker writes to disk before ACK-ing publisher. Survives restarts. Zero message loss.
◆ **At-Least-Once Delivery**	Subscriber must ACK each message. If it crashes, broker redelivers to another instance automatically.
◆ **Downtime Tolerance**	Messages queue while Notification Service is down. It processes the backlog on recovery — core banking unaffected.
◆ **Dead-Letter Queue (DLQ)**	After max retries, failed messages go to DLQ. Ops can inspect, fix, and replay — no silent loss.
◆ **Horizontal Scaling**	Add more Notification Service instances → broker distributes messages across all. Handles peak loads elastically.
◆ **Load Smoothing**	Broker is a buffer: banking system publishes bursts instantly; notification service works through them at its own pace.